



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Discover. Learn. Empower.

Experiment-1.4

Student Name: Vaibhav Kumar

UID: 23BCS13386

Branch: B.E-C.S.E

Section/Group: 23KRG-2B

Semester: 5th

Date of Performance: 19/08/2025

Subject Name: DAA

Subject Code: 23CSH-301

1. Aim: Apply the concept of Linked list and write code to Insert and Delete an element at the beginning and attend in Doubly and Circular Linked List.

2. Objective: To understand doubly and circular linked list

3. Input/Apparatus Used: Doubly and circular Linked List is used.

4. Procedure/Algorithm: Pseudocode:

Procedure for beginning of circular linked list:

Step1. Create the new node

Step2. Set the new node's next to itself (circular) Step3. If the list is empty, return new node.

Step4. Set our new node's next to the front. Step5. Set tail's next to our new node.

Step6. Return the end of the list.

Procedure for end of circular linked list:

Step1. Create the new node

Step2. Set the new node's next to itself (circular) Step3. If the list is empty, return new node.

Step4. Set our new node's next to the front. Step5. Set tail's next to our new node.

Step6. Return the end of the list.

5. Code:

```
#include <iostream>
using namespace std;

// Doubly Linked List
class Node {
public:
    int data;
    Node* prev;
    Node* next;
    Node(int val) {
        data = val;
        prev = next = NULL;
    }
};

class DoublyLinkedList {
    Node* head;
public:
    DoublyLinkedList() { head = NULL; }

    void insertAtBeginning(int val) {
        Node* newNode = new Node(val);
        if (!head) {
            head = newNode;
            return;
        }
        newNode->next = head;
        head->prev = newNode;
        head = newNode;
    }

    void insertAtEnd(int val) {
        Node* newNode = new Node(val);
        if (!head) {
            head = newNode;
            return;
        }
        Node* temp = head;
        while (temp->next) temp = temp->next;
        temp->next = newNode;
        newNode->prev = temp;
    }

    void deleteAtBeginning() {
        if (!head) return;
        Node* temp = head;
        head = head->next;
        if (head) head->prev = NULL;
        delete temp;
    }
}
```

```
void deleteAtEnd() {
    if (!head) return;
    Node* temp = head;
    if (!temp->next) {
        head = NULL;
        delete temp;
        return;
    }
    while (temp->next) temp = temp->next;
    temp->prev->next = NULL;
    delete temp;
}

void display() {
    Node* temp = head;
    while (temp) {
        cout << temp->data << " ";
        temp = temp->next;
    }
    cout << endl;
}

// Circular Linked List
class CNode {
public:
    int data;
    CNode* next;
    CNode(int val) {
        data = val;
        next = this;
    }
};
```

```
class CircularLinkedList {
    CNode* tail;
public:
    CircularLinkedList() { tail = NULL; }

    void insertAtBeginning(int val) {
        CNode* newNode = new CNode(val);
        if (!tail) {
            tail = newNode;
            return;
        }
        newNode->next = tail->next;
        tail->next = newNode;
    }

    void insertAtEnd(int val) {
        CNode* newNode = new CNode(val);
        if (!tail) {
            tail = newNode;
            return;
        }
        newNode->next = tail->next;
        tail->next = newNode;
        tail = newNode;
    }

    void deleteAtBeginning() {
        if (!tail) return;
        CNode* head = tail->next;
        if (head == tail) {
            delete head;
            tail = NULL;
            return;
        }
        tail->next = head->next;
        delete head;
    }

    void deleteAtEnd() {
        if (!tail) return;
        CNode* head = tail->next;
        if (head == tail) {
            delete head;
            tail = NULL;
            return;
        }
        CNode* temp = head;
        while (temp->next != tail) temp = temp->next;
        temp->next = tail->next;
        delete tail;
        tail = temp;
    }
}
```

```
void display() {
    if (!tail) return;
    CNode* temp = tail->next;
    do {
        cout << temp->data << " ";
        temp = temp->next;
    } while (temp != tail->next);
    cout << endl;
}

// Main Function
int main() {
    cout << "--- Doubly Linked List ---\n";
    DoublyLinkedList dll;
    dll.insertAtBeginning(10);
    dll.insertAtEnd(20);
    dll.insertAtBeginning(5);
    dll.display();
    dll.deleteAtEnd();
    dll.display();
    dll.deleteAtBeginning();
    dll.display();

    cout << "\n--- Circular Linked List ---\n";
    CircularLinkedList cll;
    cll.insertAtBeginning(10);
    cll.insertAtEnd(20);
    cll.insertAtBeginning(5);
    cll.display();
    cll.deleteAtEnd();
    cll.display();
    cll.deleteAtBeginning();
    cll.display();

    return 0;
}
```



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Discover. Learn. Empower.

```
PS C:\Users\vaibh\OneDrive\Desktop\C tutorial 2> ./DAAExp4.exe
--- Doubly Linked List ---
5 10 20
5 10
10

--- Circular Linked List ---
5 10 20
5 10
10
PS C:\Users\vaibh\OneDrive\Desktop\C tutorial 2> |
```